

MGN342-CA2

NAME – HARSHITA CHOPRA

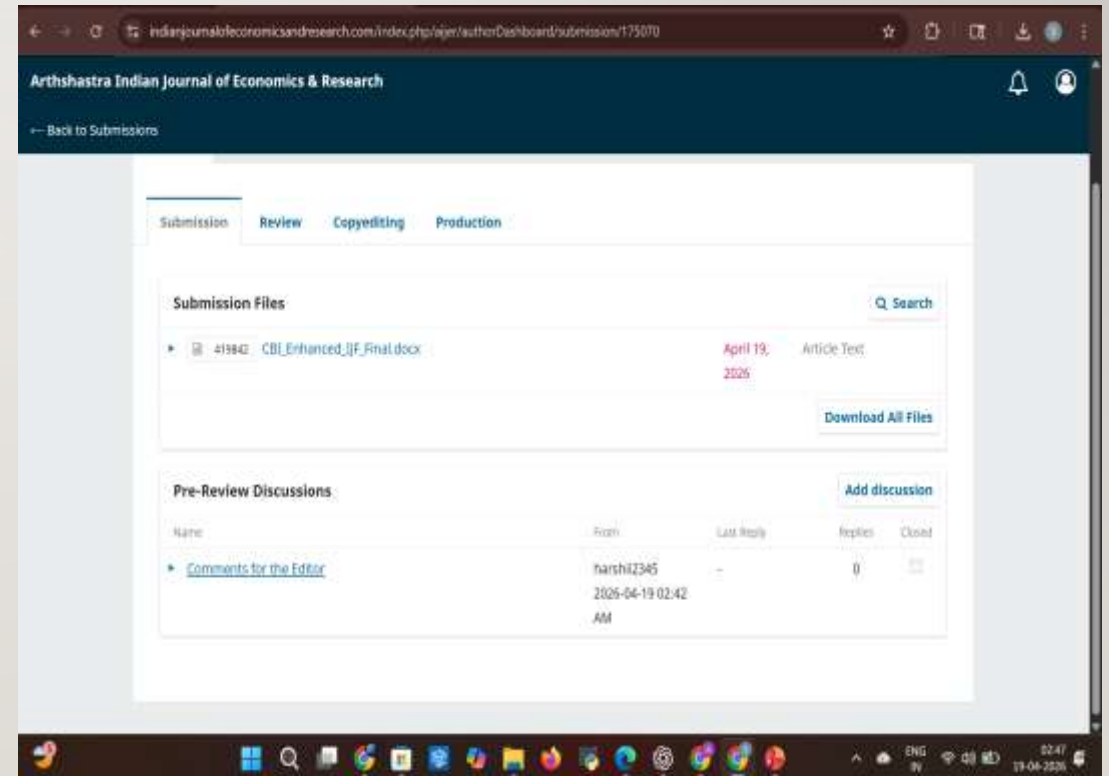
SECTION – Q2565B62

REGISTRATION NO – I25I3837



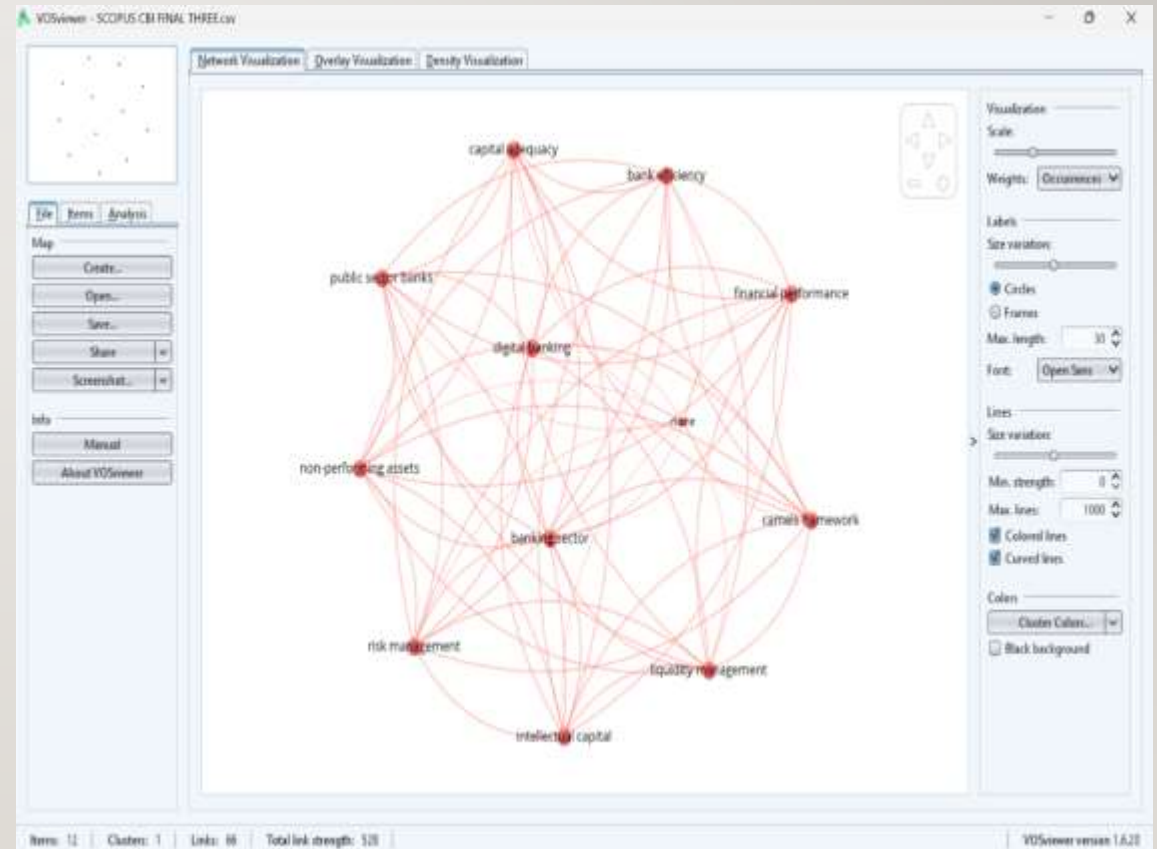
SUBMISSION REPORT

- SUBMITTED MY JOURNAL TO INDIAN JOURNALS OF ECONOMICS AND RESEARCH
- LINK - <https://indianjournalofeconomicsandresearch.com/index.php/aijer/authorDashboard/submission/175070>



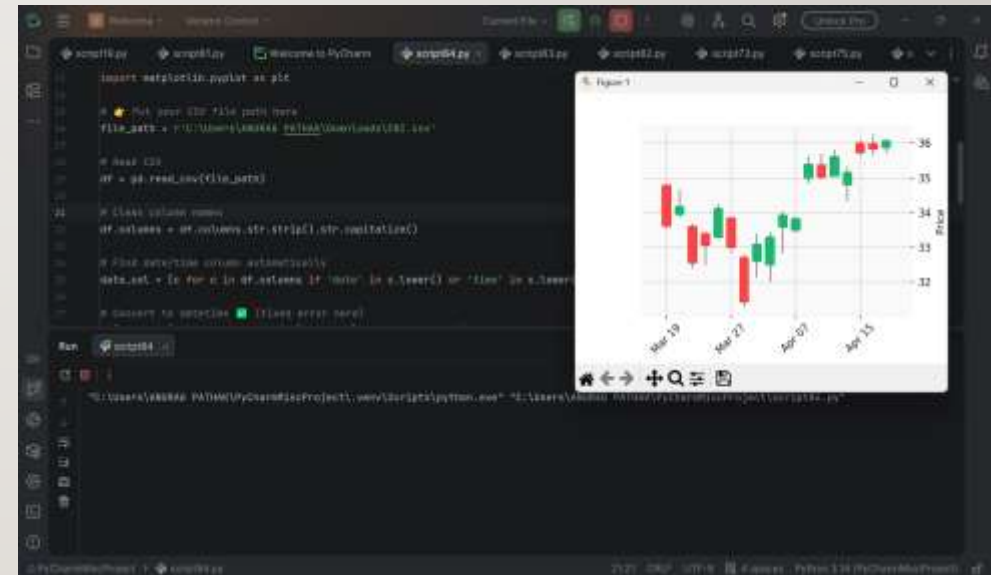
BIBLIOMETRIC IMAGE OF CBI

- The analysis revealed strong thematic interconnections within banking research, highlighting the integrated nature of performance evaluation, regulatory frameworks, and risk management in the banking sector



CANDEL STICK

- Analyzed stock price movements of Central Bank of India using candlestick charts. This visualization highlights market trends through Open, High, Low, and Close values, helping in better understanding of price behavior and investor sentiment. A great step forward in exploring financial data analytics and real-world applications of Python. #Python #DataAnalytics #StockMarket #CentralBankOfIndia #Finance #Visualization #Learning
- LINKED IN LINK -
https://www.linkedin.com/posts/harshitachopra19_python-dataanalytics-stockmarket-share-7451579058108170240-6zwM?utm_source=social_share_send&utm_medium=android_app&rcm=ACoAAF2YFwxwBwvR9-BdATV8TVzQpjMGRIUCgt24&utm_campaign=whatsapp



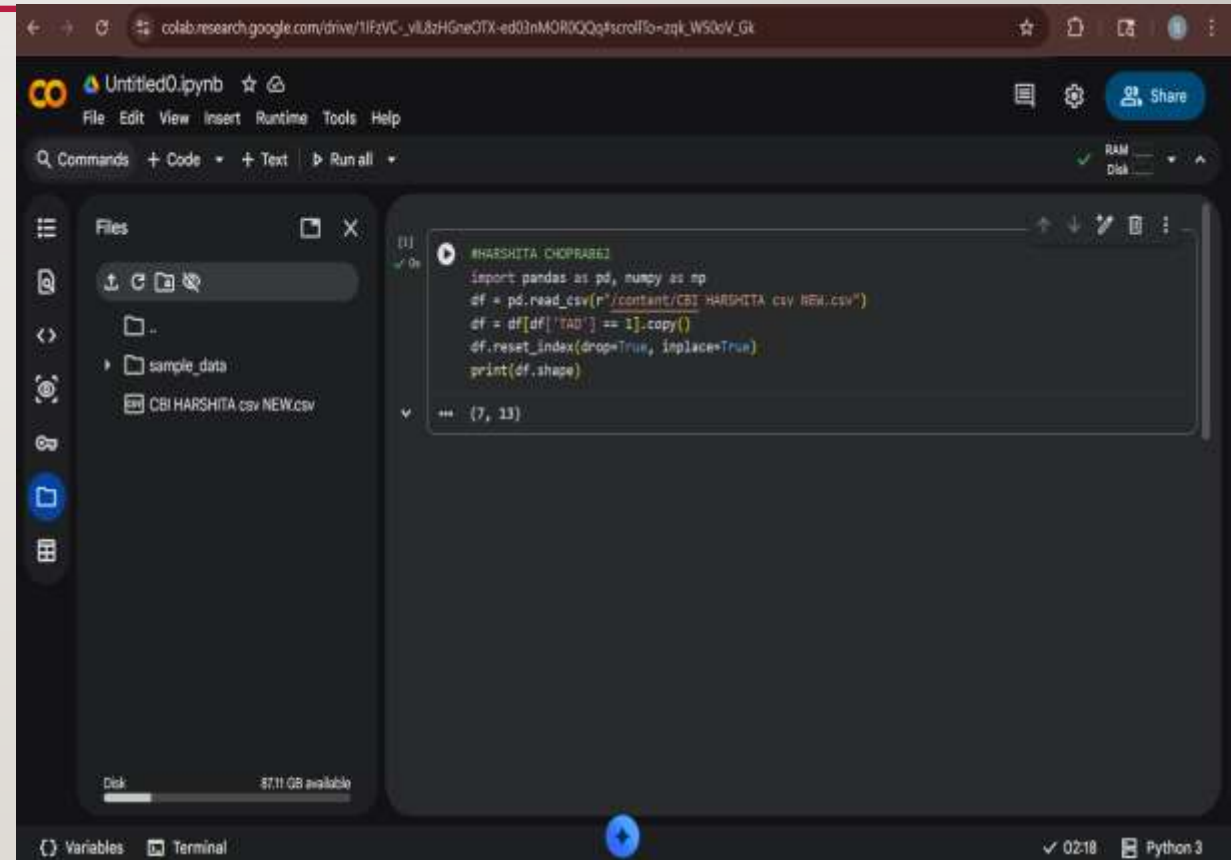
I.HOW DO WE REMOVE ROWS THAT FAILED THE ATTENTION CHECK?

- `import pandas as pd, numpy as np`
- `df =
pd.read_csv(r"C:\D4P\Tatasteel.csv")`
`)`

```
df = df[df['TAD'] == 1].copy()
```

```
df.reset_index(drop=True,  
inplace=True)
```

```
print(df.shape)
```



The screenshot shows a Google Colab notebook interface. The left sidebar displays a file explorer with a folder named 'sample_data' containing a file 'CBI HARSHITA csv NEW.csv'. The main editor area shows a Python script with the following code:

```
#HARSHITA CHOPRASEI
import pandas as pd, numpy as np
df = pd.read_csv(r"/content/CBI HARSHITA csv NEW.csv")
df = df[df['TAD'] == 1].copy()
df.reset_index(drop=True, inplace=True)
print(df.shape)
```

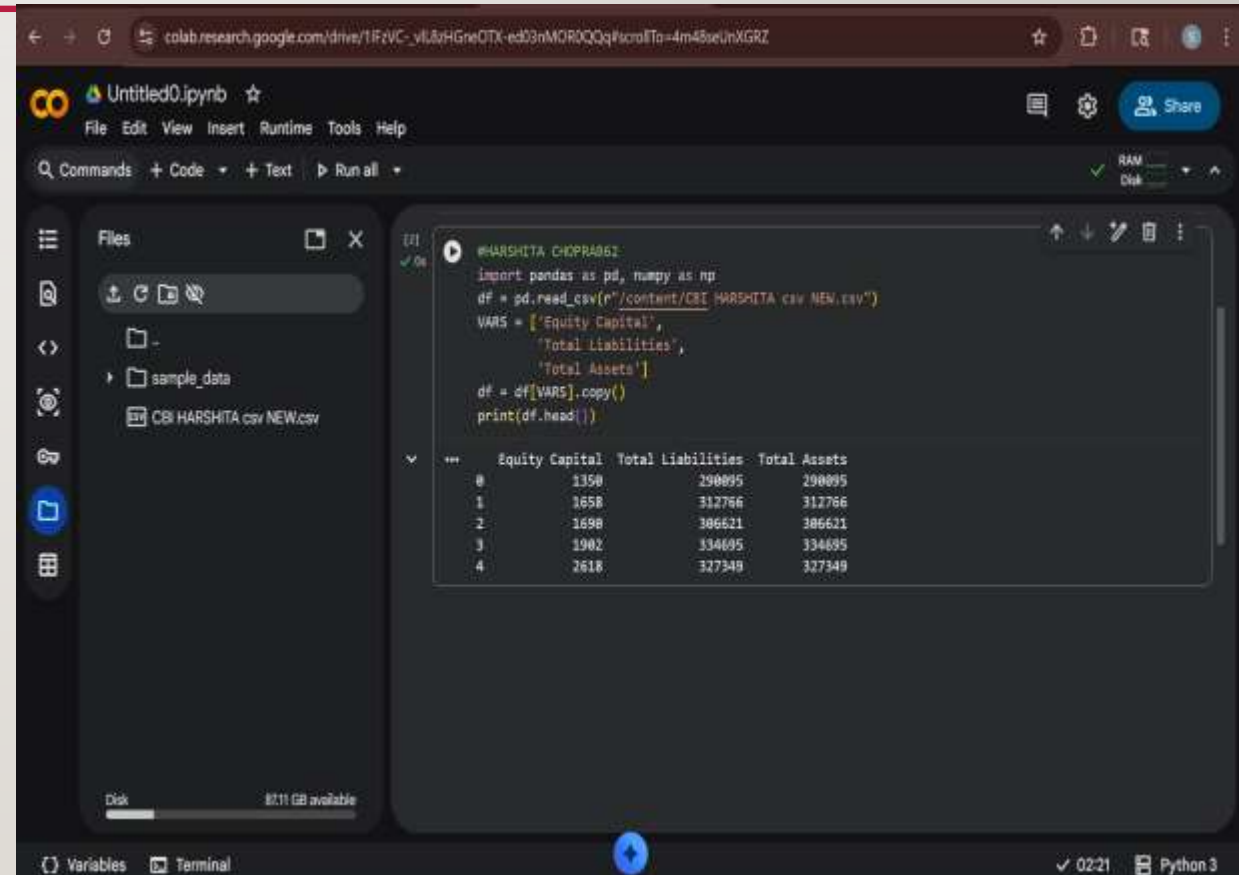
The output of the script is displayed below the code cell, showing the shape of the DataFrame as `(7, 13)`. The bottom status bar indicates the notebook is running on Python 3.

2.HOW DO WE SELECT ONLY THE VARIABLES WE NEED?

- `import pandas as pd, numpy as np`
- `df =
pd.read_csv(r"C:\D4P\Tatasteel.csv")`

```
VARs = ['EquityCapital',  
        'TotalLiabilities',  
        'TotalAssets']
```

```
df = df[VARs].copy()  
print(df.head())
```



The screenshot shows a Google Colab notebook titled 'Untitled0.ipynb'. The code cell contains the following Python code:

```
#HARSHITA CHOUPRA062  
import pandas as pd, numpy as np  
df = pd.read_csv(r"/content/CBI HARSHITA csv NEW.csv")  
VARs = ['Equity Capital',  
        'Total Liabilities',  
        'Total Assets']  
df = df[VARs].copy()  
print(df.head())
```

The output of the code is a table showing the first five rows of the filtered data:

	Equity Capital	Total Liabilities	Total Assets
0	1350	290095	290095
1	1658	312766	312766
2	1698	306621	306621
3	1902	334695	334695
4	2618	327349	327349

The interface includes a file explorer on the left showing a folder named 'sample_data' and a file named 'CBI HARSHITA csv NEW.csv'. The bottom status bar indicates '02:21' and 'Python 3'.

3.HOW DO WE DELETE ROWS WHERE MORE THAN 50% OF VALUES ARE MISSING?

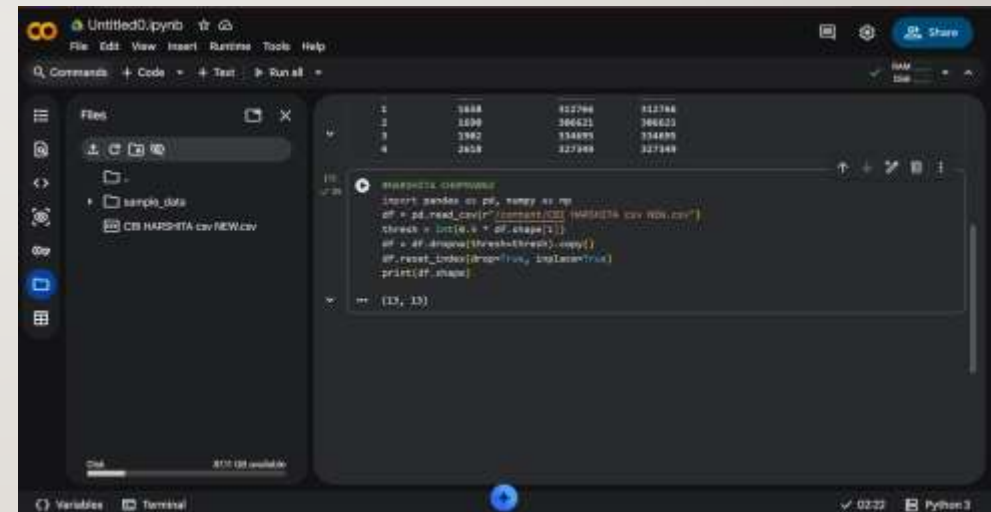
- `import pandas as pd, numpy as np`
- `df =
pd.read_csv(r"C:\D4P\Tatasteel.csv")`

```
thresh = int(0.5 * df.shape[1])
```

```
df = df.dropna(thresh=thresh).copy()
```

```
df.reset_index(drop=True,  
inplace=True)
```

```
print(df.shape)
```



4. HOW DO WE FILL MISSING NUMERIC VALUES WITH THE COLUMN MEAN?

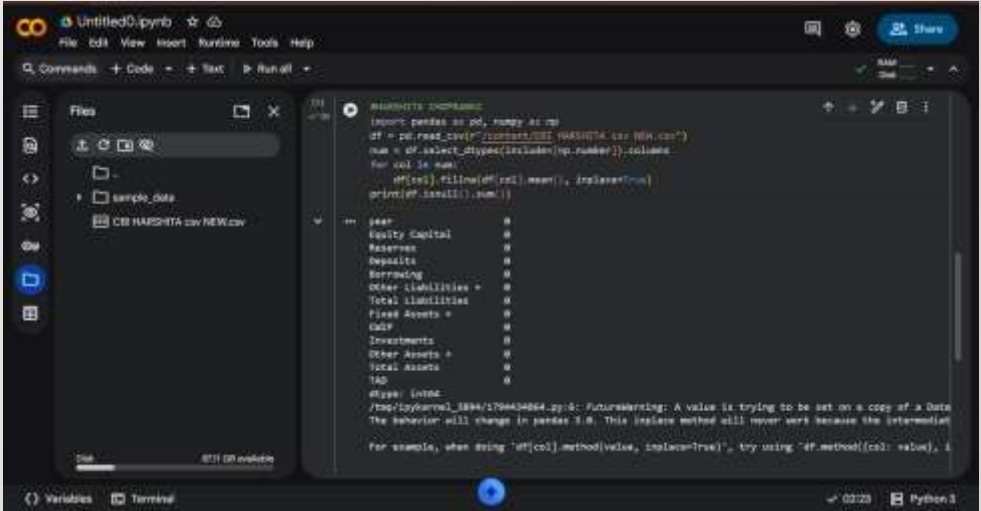
```
• import pandas as pd, numpy as np

• df =
  pd.read_csv(r"C:\D4P\Tatasteel.csv")

num =
df.select_dtypes(include=[np.number]).columns

for col in num:
    df[col].fillna(df[col].mean(),
inplace=True)

print(df.isnull().sum())
```



The screenshot shows a Jupyter Notebook interface with a dark theme. The code in the cell is as follows:

```
import pandas as pd, numpy as np
df = pd.read_csv(r"C:\D4P\Tatasteel.csv")
num = df.select_dtypes(include=[np.number]).columns
for col in num:
    df[col].fillna(df[col].mean(), inplace=True)
print(df.isnull().sum())
```

The output of the code is a series of zeros, indicating that all missing values have been successfully filled. The output is displayed in a table-like format with columns for each variable and a final column for the sum of null values.

Variable	Sum
year	0
Equity Capital	0
Reserves	0
Deposits	0
Borrowing	0
Other liabilities	0
Total liabilities	0
Fixed Assets	0
CAF	0
Investments	0
Other Assets	0
Total Assets	0
TAD	0

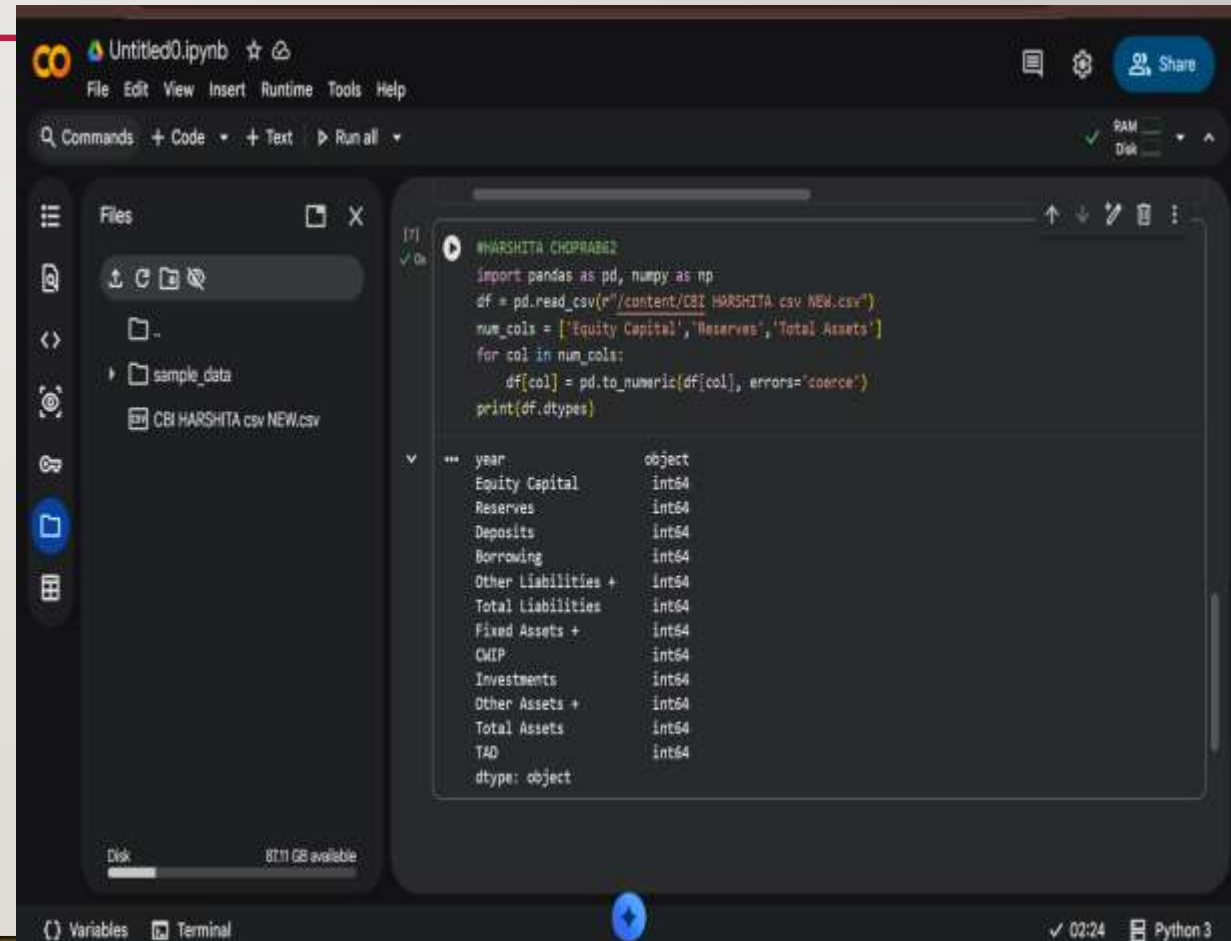
At the bottom of the output, there is a warning message: "FutureWarning: A value is trying to be set on a copy of a Data. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate For example, when using 'df[col].method(value, inplace=True)', try using 'df.method(col, value)'."

5. HOW DO WE ENSURE ALL NUMERIC COLUMNS HAVE THE CORRECT DATA TYPE?

```
• import pandas as pd, numpy as np
• df =
  pd.read_csv(r"C:\D4P\Tatasteel.csv")

num_cols =
['EquityCapital', 'Reserves', 'TotalAssets']

for col in num_cols:
    df[col] = pd.to_numeric(df[col],
errors='coerce')
print(df.dtypes)
```



The screenshot shows a Jupyter Notebook interface with a dark theme. The left sidebar displays a file explorer with a folder named 'sample_data' containing a file 'CBI HARSHITA csv NEW.csv'. The main area shows a code cell with the following Python code:

```
#HARSHITA CHOFRABEZ
import pandas as pd, numpy as np
df = pd.read_csv(r"/content/CBI HARSHITA csv NEW.csv")
num_cols = ['Equity Capital', 'Reserves', 'Total Assets']
for col in num_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')
print(df.dtypes)
```

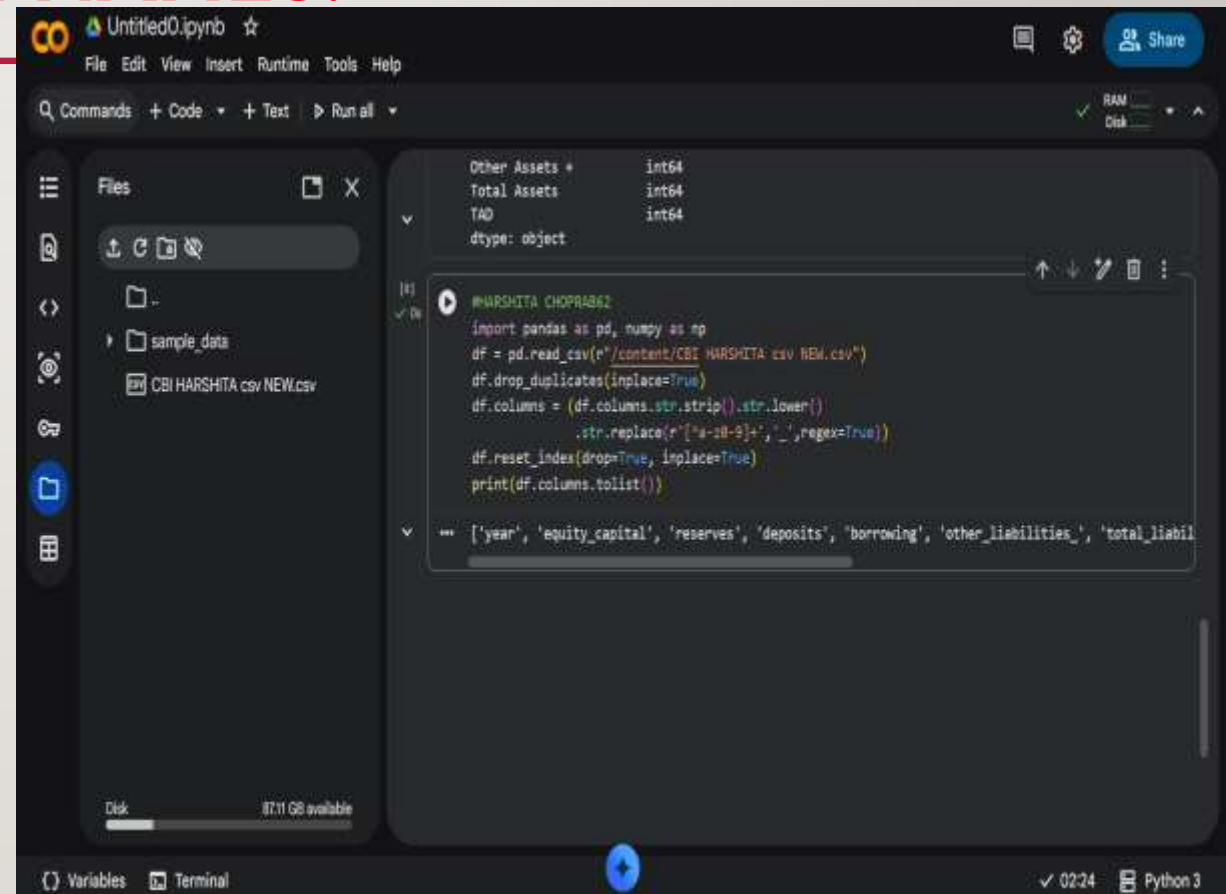
Below the code cell, the output of the code is displayed as a table:

year	object
Equity Capital	int64
Reserves	int64
Deposits	int64
Borrowing	int64
Other Liabilities +	int64
Total Liabilities	int64
Fixed Assets +	int64
CHMP	int64
Investments	int64
Other Assets +	int64
Total Assets	int64
TAD	int64
dtype:	object

The bottom status bar indicates 'Disk 87.11 GB available', 'Variables', 'Terminal', and 'Python 3'.

6. HOW DO WE REMOVE DUPLICATES AND STANDARDISE COLUMN NAMES?

```
• import pandas as pd, numpy as np
• df =
  pd.read_csv(r"C:\D4P\Tatasteel.csv")
df.drop_duplicates(inplace=True)
df.columns =
(df.columns.str.strip().str.lower()
    .str.replace(r'[^a-z0-9]+' , '_' , regex=True))
df.reset_index(drop=True, inplace=True)
print(df.columns.tolist())
```



Untitled0.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- sample_data
- CBI HARSHITA csv NEW.csv

Other Assets + int64
Total Assets int64
TAO int64
dtype: object

```
[1]:
import pandas as pd, numpy as np
df = pd.read_csv(r"/content/CBI HARSHITA csv NEW.csv")
df.drop_duplicates(inplace=True)
df.columns = (df.columns.str.strip().str.lower()
    .str.replace(r'[^a-z0-9]+' , '_' , regex=True))
df.reset_index(drop=True, inplace=True)
print(df.columns.tolist())
```

['year', 'equity_capital', 'reserves', 'deposits', 'borrowing', 'other_liabilities_', 'total_liabil

02:24 Python 3

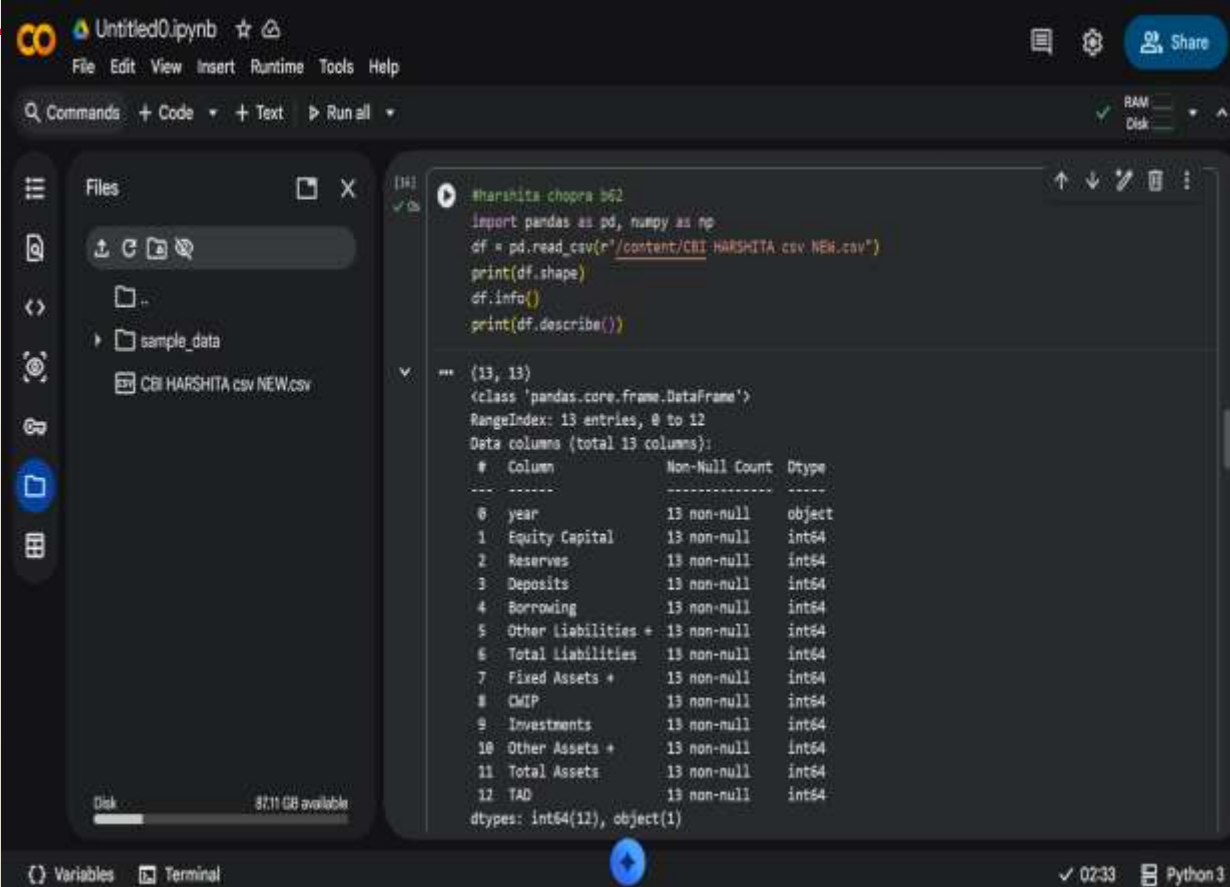
7.HOW DO WE GET A SUMMARY OF THE DATASET'S STRUCTURE?

- `import pandas as pd, numpy as np`
- `df = pd.read_csv(r"C:\D4P\Tatasteel.csv")`

`print(df.shape)`

`df.info()`

`print(df.describe())`



The screenshot shows a Jupyter Notebook interface with a dark theme. The left sidebar displays a file explorer with a folder named 'sample_data' containing a file 'CBI HARSHITA csv NEW.csv'. The main area shows a code cell with the following Python code:

```
#harshita chopra b62
import pandas as pd, numpy as np
df = pd.read_csv(r"/content/CBI HARSHITA csv NEW.csv")
print(df.shape)
df.info()
print(df.describe())
```

The output of the code is displayed below the cell:

```
(13, 13)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13 entries, 0 to 12
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   year                  13 non-null    object
1   Equity Capital         13 non-null    int64
2   Reserves               13 non-null    int64
3   Deposits               13 non-null    int64
4   Borrowing              13 non-null    int64
5   Other liabilities +    13 non-null    int64
6   Total liabilities      13 non-null    int64
7   Fixed Assets +        13 non-null    int64
8   CWP                   13 non-null    int64
9   Investments            13 non-null    int64
10  Other Assets +        13 non-null    int64
11  Total Assets           13 non-null    int64
12  TAD                   13 non-null    int64
dtypes: int64(12), object(1)
```

The bottom status bar indicates '02:33' and 'Python 3'.

8.HOW DO WE COMPUTE MEAN, MEDIAN, MIN, MAX AND STD DEVIATION?

```
• import pandas as pd, numpy as np

• df = pd.read_csv(r"C:\D4P\Tatasteel.csv")

num =
df.select_dtypes(include=[np.number]).columns

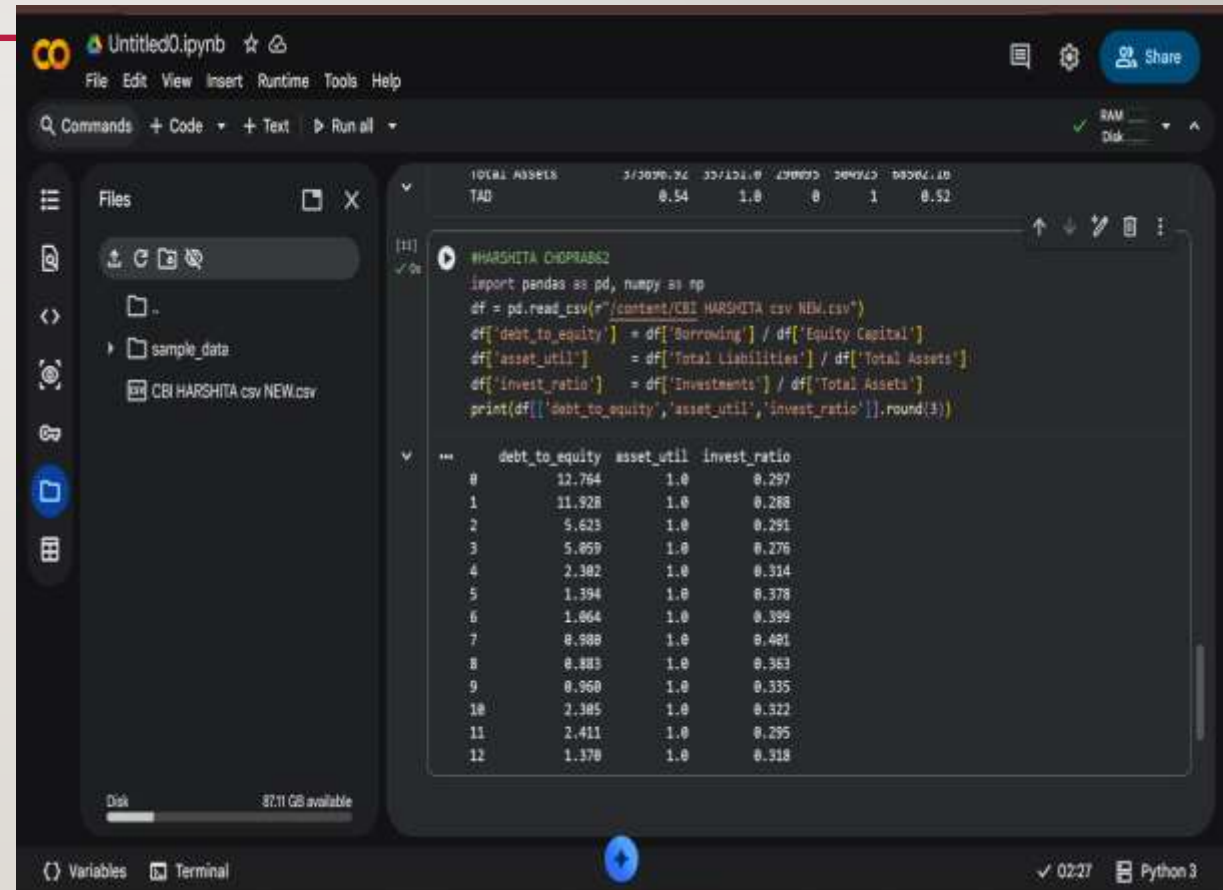
stats = pd.DataFrame({

    'Mean':df[num].mean(),
    'Median':df[num].median(),

    'Min':df[num].min(),
    'Max':df[num].max(),

    'Std':df[num].std()})

print(stats.round(2))
```



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The code editor contains a Python script that reads a CSV file, calculates statistical measures (Mean, Median, Min, Max, Std) for numerical columns, and prints the results. The output shows a DataFrame with columns 'debt_to_equity', 'asset_util', and 'invest_ratio'.

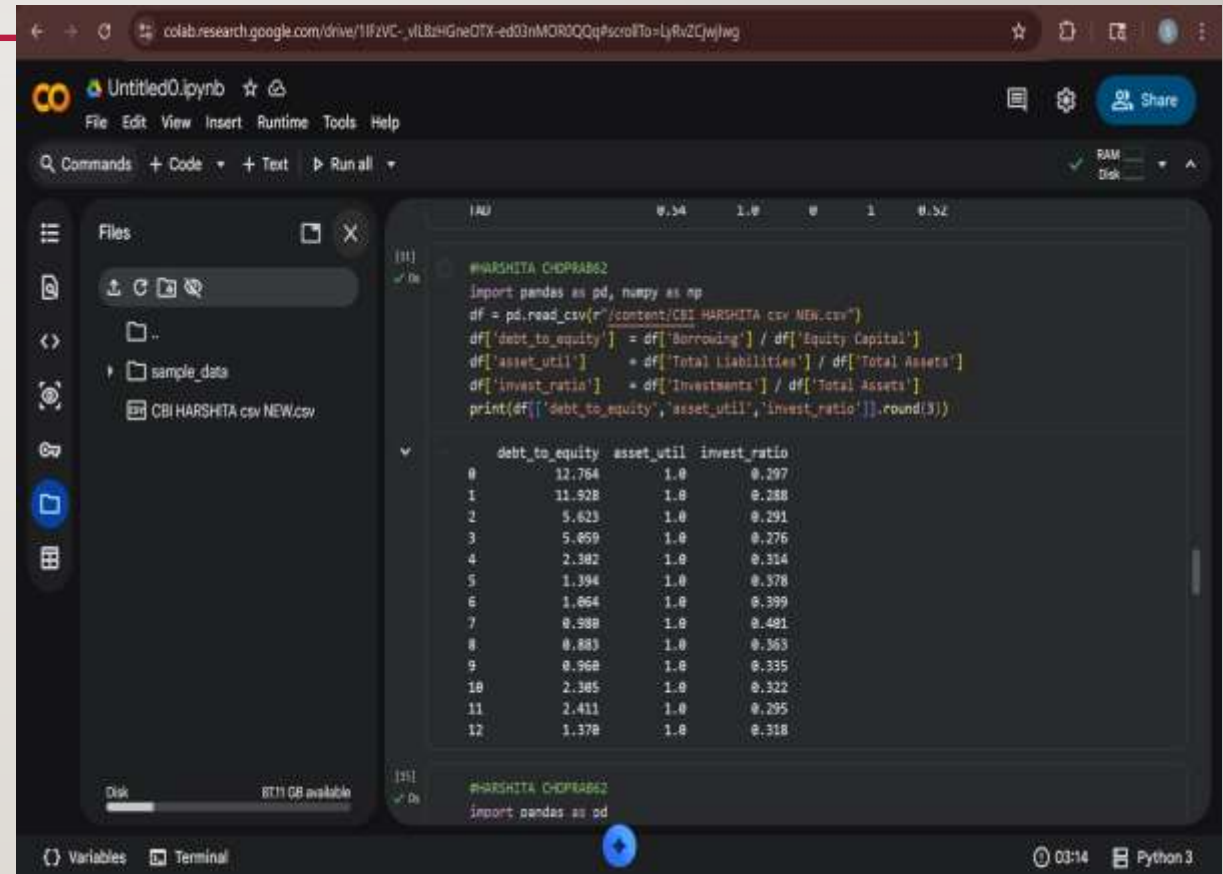
```
import pandas as pd, numpy as np
df = pd.read_csv(r"/content/CBI HARSHITA csv NEW.csv")
df['debt_to_equity'] = df['Borrowing'] / df['Equity Capital']
df['asset_util'] = df['Total Liabilities'] / df['Total Assets']
df['invest_ratio'] = df['Investments'] / df['Total Assets']
print(df[['debt_to_equity', 'asset_util', 'invest_ratio']].round(3))
```

	debt_to_equity	asset_util	invest_ratio
0	12.764	1.0	0.297
1	11.928	1.0	0.288
2	5.623	1.0	0.291
3	5.059	1.0	0.276
4	2.382	1.0	0.314
5	1.394	1.0	0.378
6	1.064	1.0	0.399
7	0.988	1.0	0.481
8	0.883	1.0	0.363
9	0.960	1.0	0.335
10	2.385	1.0	0.322
11	2.411	1.0	0.295
12	1.378	1.0	0.318

9. HOW DO WE CREATE NEW DERIVED COLUMNS FROM EXISTING ONES?

- `import pandas as pd, numpy as np`
- `df = pd.read_csv(r"C:\D4P\Tatasteel.csv")`

```
df['debt_to_equity'] = df['Borrowings +'] /  
df['EquityCapital']  
  
df['asset_util'] =  
df['TotalLiabilities'] / df['TotalAssets']  
  
df['invest_ratio'] = df['Investments'] /  
df['TotalAssets']  
  
print(df[['debt_to_equity', 'asset_util', 'invest_ratio']].round(3))
```



The screenshot shows a Google Colab notebook titled 'Untitled0.ipynb'. The code in the notebook is as follows:

```
#HARSHITA CHOPRA862  
import pandas as pd, numpy as np  
df = pd.read_csv(r"/content/CBI HARSHITA csv NEW.csv")  
df['debt_to_equity'] = df['Borrowing'] / df['Equity Capital']  
df['asset_util'] = df['Total Liabilities'] / df['Total Assets']  
df['invest_ratio'] = df['Investments'] / df['Total Assets']  
print(df[['debt_to_equity', 'asset_util', 'invest_ratio']].round(3))
```

The output of the code is a table with 13 rows and 3 columns: `debt_to_equity`, `asset_util`, and `invest_ratio`. The values are rounded to 3 decimal places.

	debt_to_equity	asset_util	invest_ratio
0	12.764	1.0	0.297
1	11.928	1.0	0.288
2	5.623	1.0	0.291
3	5.659	1.0	0.276
4	2.382	1.0	0.314
5	1.394	1.0	0.378
6	1.864	1.0	0.399
7	0.988	1.0	0.481
8	0.883	1.0	0.363
9	0.968	1.0	0.335
10	2.385	1.0	0.322
11	2.411	1.0	0.295
12	1.378	1.0	0.318

10. HOW DO WE FILTER ROWS BASED ON A CONDITION?

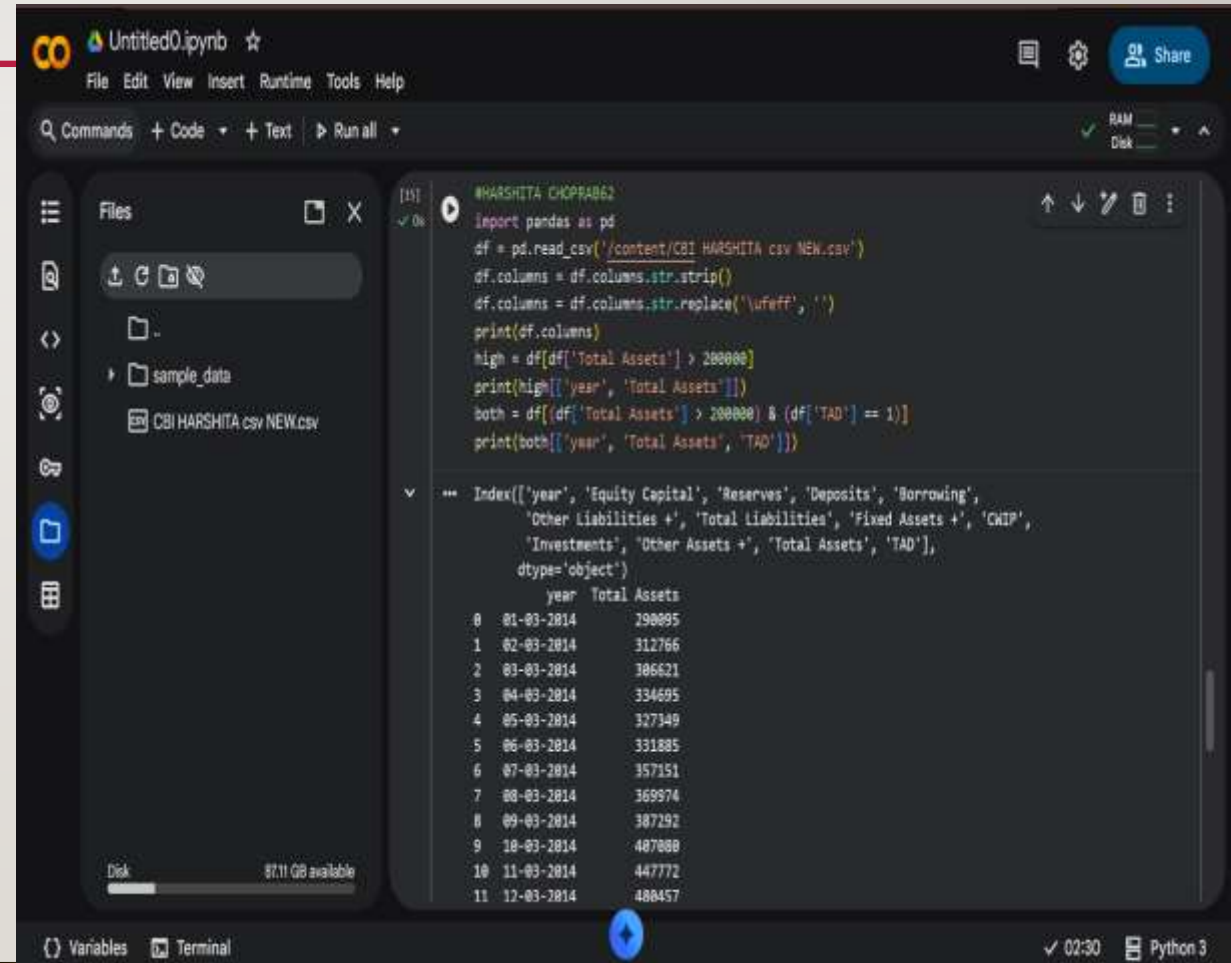
- `import pandas as pd, numpy as np`
- `df = pd.read_csv(r"C:\D4P\Tatasteel.csv")`

```
high = df[df['TotalAssets'] > 200000]
```

```
print(high[['Year', 'TotalAssets']])
```

```
both = df[(df['TotalAssets'] > 200000) &  
(df['TAD'] == 1)]
```

```
print(both[['Year', 'TotalAssets', 'TAD']])
```



The screenshot shows a Jupyter Notebook titled 'Untitled0.ipynb'. The code cell contains the following Python code:

```
#HARSHITA CHOPRA862  
import pandas as pd  
df = pd.read_csv('/content/CBI HARSHITA csv NEW.csv')  
df.columns = df.columns.str.strip()  
df.columns = df.columns.str.replace('\uffff', '')  
print(df.columns)  
high = df[df['Total Assets'] > 200000]  
print(high[['year', 'Total Assets']])  
both = df[(df['Total Assets'] > 200000) & (df['TAD'] == 1)]  
print(both[['year', 'Total Assets', 'TAD']])
```

The output of the code is displayed below the cell, showing the index of the DataFrame and a table of the filtered data:

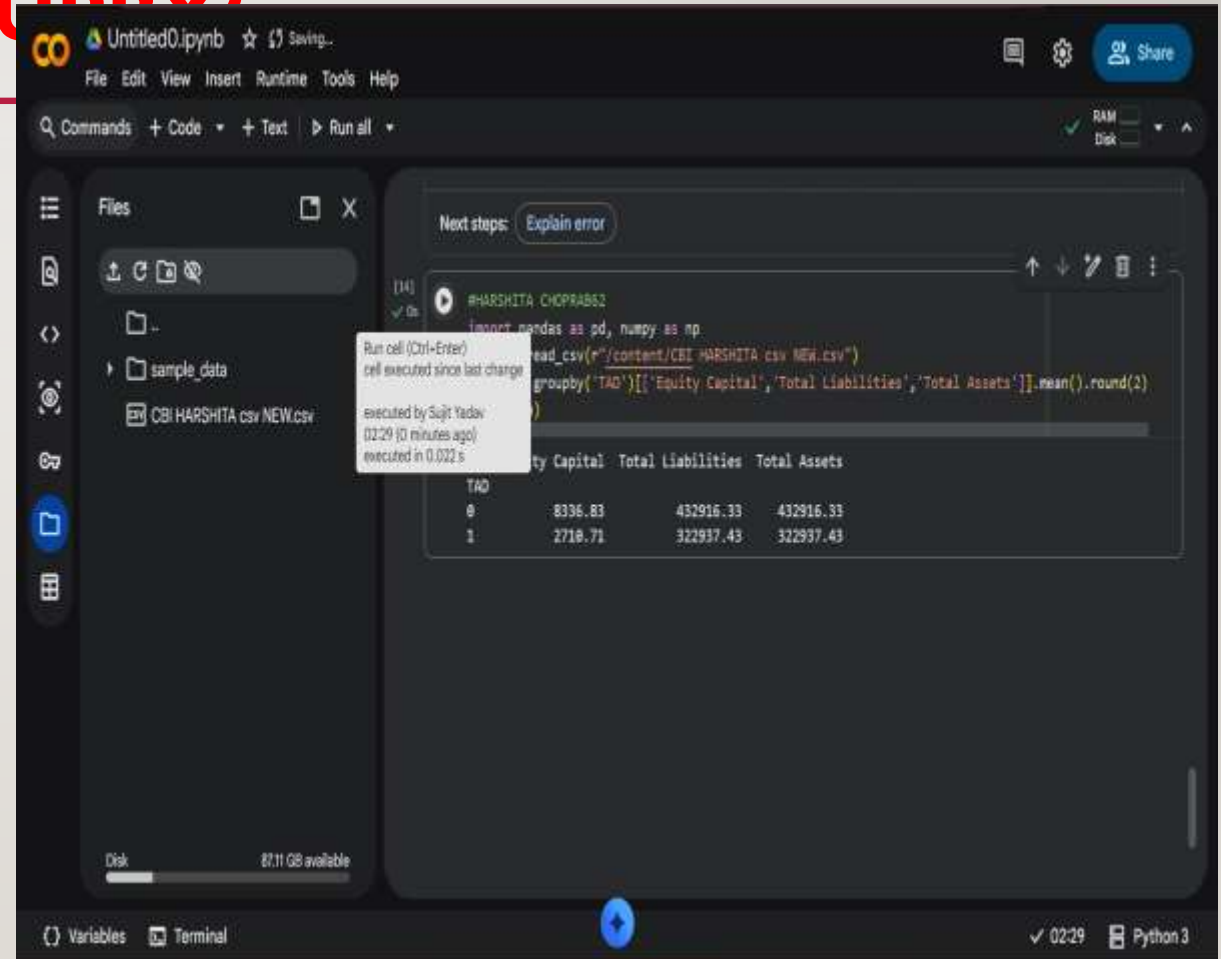
```
Index(['year', 'Equity Capital', 'Reserves', 'Deposits', 'Borrowing',  
      'Other Liabilities +', 'Total liabilities', 'Fixed Assets +', 'CWIP',  
      'Investments', 'Other Assets +', 'Total Assets', 'TAD'],  
      dtype='object')
```

	year	Total Assets
0	01-03-2014	290095
1	02-03-2014	312766
2	03-03-2014	306621
3	04-03-2014	334695
4	05-03-2014	327349
5	06-03-2014	331885
6	07-03-2014	357351
7	08-03-2014	369974
8	09-03-2014	307292
9	10-03-2014	407000
10	11-03-2014	447772
11	12-03-2014	480457

11. HOW DO WE COMPUTE GROUP-WISE STATISTICS USING GROUPBY?

- `import pandas as pd, numpy as np`
- `df =`
`pd.read_csv(r"C:\D4P\Tatasteel.csv")`

```
grp =  
df.groupby('TAD')[['EquityCapital', 'TotalLiabilities', 'TotalAssets']].mean().round(2)  
print(grp)
```



The screenshot shows a Jupyter Notebook environment with a file explorer on the left, a code editor in the center, and a console/output area on the right. The code in the cell is as follows:

```
[14]:  
import pandas as pd, numpy as np  
df = pd.read_csv(r"C:\content\CBI HARSHITA csv NEW.csv")  
grp = df.groupby('TAD')[['Equity Capital', 'Total Liabilities', 'Total Assets']].mean().round(2)
```

The output of the code is a pandas DataFrame showing the mean values for each 'TAD' group, rounded to 2 decimal places:

TAD	Equity Capital	Total Liabilities	Total Assets
0	8336.83	432916.33	432916.33
1	2718.71	322937.43	322937.43

The interface also shows a 'Next steps: Explain error' button and a 'Run cell (Ctrl-Enter)' tooltip.